

# Storage Wars

Postgres 19 cuts five features to ship safely, and MongoDB 9.0 changes what null means

Issue #1

Mon, 05 Oct 2026

~34 min read

Level 3/5

## Today's two stories

1. **PostgreSQL 19 is weeks from release, and it's lighter than promised.** Graph queries, online checksums and temporal updates were pulled in Beta 4. What's left (REPACK, WAIT FOR LSN, planner advice, smarter autovacuum) is still a big release, with defaults you need to check before you upgrade.
2. **MongoDB 9.0 is GA.** Behind the "up to 2x throughput" headline are changes that can quietly alter your query results ( `null` on dotted paths), new guardrails that can kill heavy queries, and a new release model for self-managed users.
3. **On the radar:** 6 more things that happened, listed at the end.

## STORY 1 PostgreSQL 19: the release that got smaller on purpose

POSTGRES 19 · BETA 4 ON 24 SEP 2026 · RC DUE EARLY OCTOBER · ~18 MIN READ

In June, the PostgreSQL 19 Beta 1 announcement promised graph queries written in standard SQL, the ability to switch on data checksums without a restart, and `UPDATE ... FOR PORTION OF` for time-ranged data. On 24 September, Beta 4 shipped without any of them. Nothing broke in production and nobody shipped a buggy release. The project read its own bug tracker, decided those features weren't ready, and took them out. That decision tells you a lot about why people trust Postgres with their data. And what survived is still a release worth planning for.

### WHAT HAPPENED

PostgreSQL 19 Beta 4 came out on **24 September 2026**. Its announcement lists features that were **reverted**, meaning their code was removed from the release branch: SQL/PGQ property graph queries, online enabling and disabling of data checksums, temporal `UPDATE / DELETE ... FOR PORTION OF`, `ALTER TABLE ... MERGE PARTITIONS` and `SPLIT PARTITIONS`, and three functions that generate DDL ( `pg_get_role_ddl()`, `pg_get_tablespace_ddl()` and `pg_get_database_ddl()` ).<sup>1</sup>

The project explained the cuts plainly: Postgres "must be reliable", so the community removed some features from 19 "so they can be potentially be included in a later major release after more time working through the community development and review process."<sup>1</sup>

On timing, Beta 4 says the next release is the release candidate, which "should occur in early October", and that GA "may also occur in October".<sup>1</sup> The project roadmap lists PostgreSQL 19 as "planned for October 2026".<sup>2</sup> The draft release notes still show the release date as 2026-??-??.<sup>3</sup>

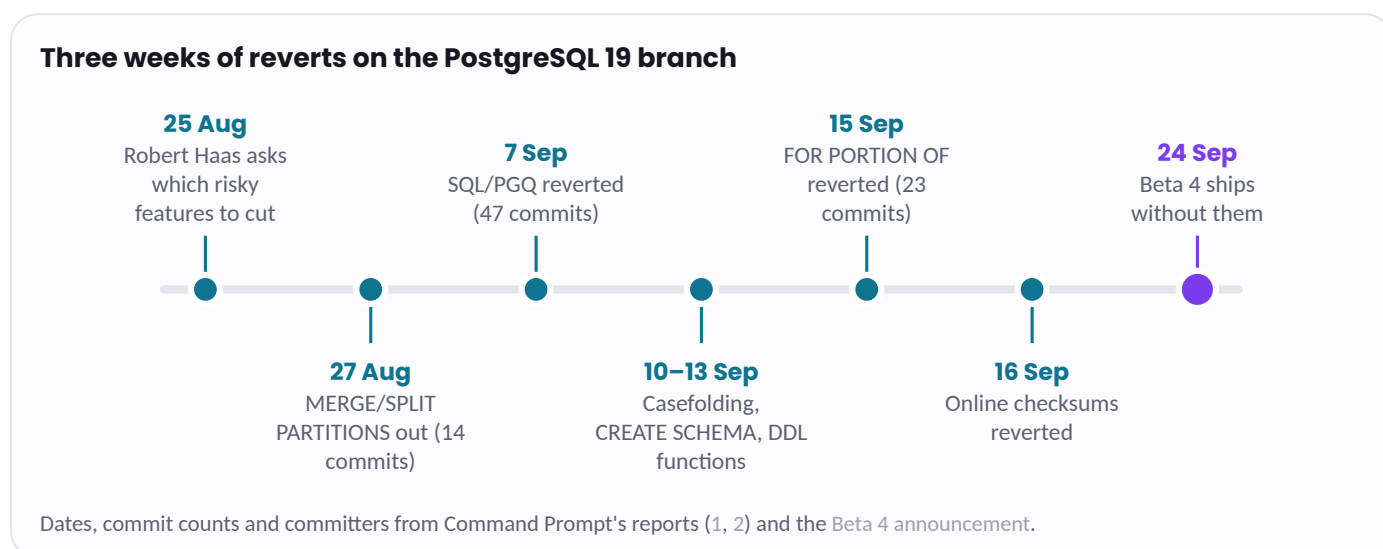
That makes 19 a little later than usual. PostgreSQL 18 shipped on 25 September 2025, three weeks after its first release candidate.<sup>4,5</sup> But October isn't unheard of: PostgreSQL 15 came out on 13 October 2022.<sup>5</sup>

## THE BACKGROUND YOU NEED

**How a Postgres release is made.** The PostgreSQL Global Development Group ships one major version *≈ a release with new features that can change on-disk formats, so upgrading needs `pg_upgrade` or a dump and restore* about once a year, and supports each one for **five years** with minor releases *≈ bug- and security-fix releases such as 18.6 that you can install by just swapping binaries* at least every three months.<sup>5</sup> New features land in the development branch all year. After the feature freeze, the branch goes through public **betas**, then a **release candidate (RC)** that's meant to be identical to the final release unless a bug turns up, and then **general availability (GA)**.

The betas exist so that people outside the core team can find bugs before the features reach production. When a feature keeps needing fixes after it has been committed, the community can **revert** it: undo its commits and try again in a later release. That's what happened in 19, on an unusual scale.

**How the cuts happened.** On 25 August, core developer Robert Haas asked on the `pgsql-hackers` mailing list whether six heavily patched features should be removed before 19 shipped.<sup>6</sup> Over the next three weeks, committers removed them one by one:



The reasons were concrete. SQL/PGQ and the partition commands had "multiple design issues which are too late to address in this release cycle".<sup>6</sup> Online checksums had "a number of postcommit fixes", which raised the risk that more bugs would surface after release.<sup>7</sup> Two `CREATE SCHEMA` changes were dropped because the "compatibility breakage" outweighed the gain, and a change to how identifiers are case-folded caused an "unintended behaviour difference" with some locales.<sup>7</sup>

**What the pulled features were.** These are worth knowing, because several will probably come back in PostgreSQL 20:

- **SQL/PGQ** is the SQL standard's syntax for **property graph queries** *≈ queries that treat rows as nodes and relationships as edges, so "friends of friends who bought X" becomes a path pattern instead of a chain of self-joins* . It would have let you query relational tables as a graph without a separate graph database.
- **Online data checksums**. A **data checksum** *≈ a small fingerprint Postgres stores in each data page and verifies on read, so silent disk corruption shows up as an error instead of wrong data* catches storage corruption. Since PostgreSQL 18, **initdb** enables checksums by default for new clusters.<sup>8</sup> Older clusters don't have them, and turning them on means running the **pg\_checksums** tool with the server stopped. That means downtime and a restart, which is painful for big databases.<sup>8</sup> There's a second snag: **pg\_upgrade** requires the old and new clusters to have the **same** checksum setting.<sup>8</sup> The online feature would have removed that downtime. For now, the usual workaround remains: enable checksums on a replica and fail over to it.<sup>8</sup>
- **Temporal FOR PORTION OF** would have let you update or delete only part of a row's validity period (for example, "this price applies from 1 to 15 March") and have Postgres split the row for you.
- **MERGE / SPLIT PARTITIONS** would have combined or divided partitions of a **partitioned table** *≈ one logical table stored as several physical child tables, each holding a range or list of keys* with one **ALTER TABLE** command.

## HOW IT WORKS: WHAT'S STILL IN POSTGRESQL 19

The official release notes list six headline features.<sup>3</sup> Here are the ones you're most likely to use, and how they work.

### 1. REPACK: VACUUM FULL and CLUSTER, renamed and made concurrent

Postgres uses **MVCC** *≈ multi-version concurrency control: an UPDATE writes a new row version and leaves the old one in place for transactions that may still need to see it* . Old versions become **dead tuples** *≈ row versions no transaction can see any more* . Regular **VACUUM** marks their space reusable but rarely shrinks the file, so a table that once held a lot of data can stay large on disk. That extra space is called **bloat** *≈ disk space a table or index occupies beyond what its live rows need* .

Until now there were two tools for rewriting a table compactly: **VACUUM FULL** and **CLUSTER** (which also sorts rows by an index). Both take an **ACCESS EXCLUSIVE lock** *≈ the strongest table lock in Postgres: it blocks every other read and write on the table until it's released* for the whole rewrite, so on a big table they mean downtime.

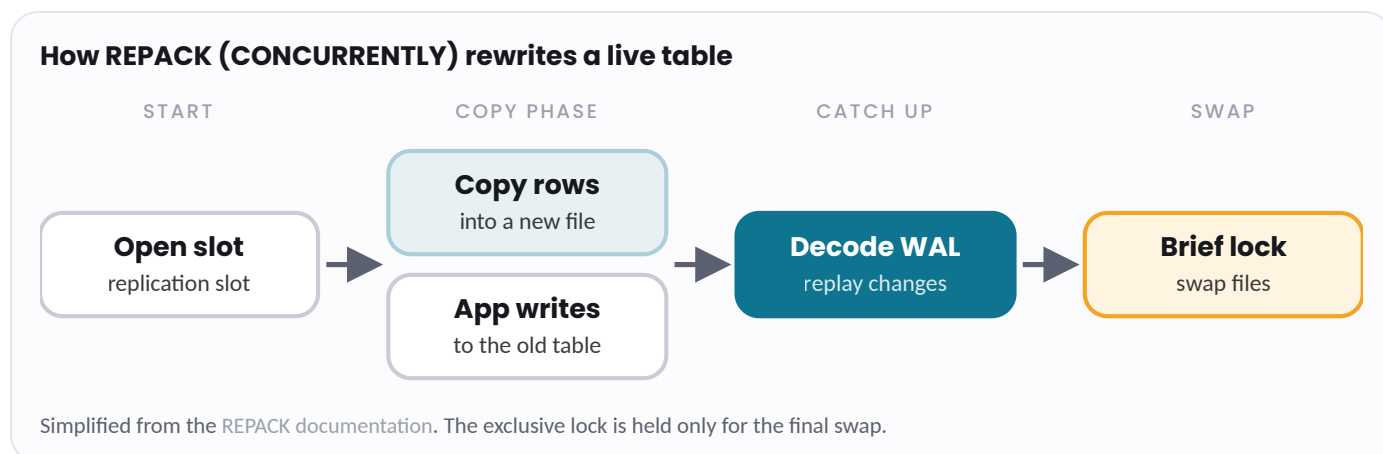
PostgreSQL 19 merges both into one command, **REPACK** . The commit message says this functionality is "completely different from regular VACUUM", so giving it its own name should make it easier to understand.<sup>9</sup> The old commands remain for compatibility.<sup>3</sup>

```
-- Rewrite one table compactly, without blocking reads and writes for most of the run
REPACK (CONCURRENTLY, VERBOSE) orders;

-- Rewrite and physically order rows by an index (what CLUSTER did)
REPACK orders USING INDEX orders_created_at_idx;
```

Plain **REPACK** still takes an **ACCESS EXCLUSIVE lock** for the whole run. The interesting part is **CONCURRENTLY** . It builds the new copy of the table while your application keeps writing to the old one. It captures those

concurrent changes with [logical decoding](#) *≈ reading the write-ahead log and turning it back into row-level changes (insert this row, update that one)*, the same machinery logical replication uses, replays them onto the new copy, and only takes the exclusive lock for the final swap.<sup>10</sup>



The catch-up step is why there are limits. `CONCURRENTLY` needs a free [replication slot](#) *≈ a bookmark in the WAL that stops Postgres from deleting log segments a consumer hasn't read yet* (the new `max_repack_replication_slots` setting controls how many are available).<sup>3</sup> It doesn't work on tables without a primary key, on unlogged or partitioned tables, on materialized views or on system catalogs, and not inside a transaction block. It can fail if someone runs DDL on the table mid-run. The docs also warn that it is "not MVCC-safe".<sup>10</sup> In practice, that means a long-running transaction that started before the repack may see the table differently afterwards. Run it when no long-running transactions are using that table.

Plan your disk space too. A rewrite needs free space roughly equal to the table plus its indexes, and about double the table size if it sorts.<sup>10</sup>

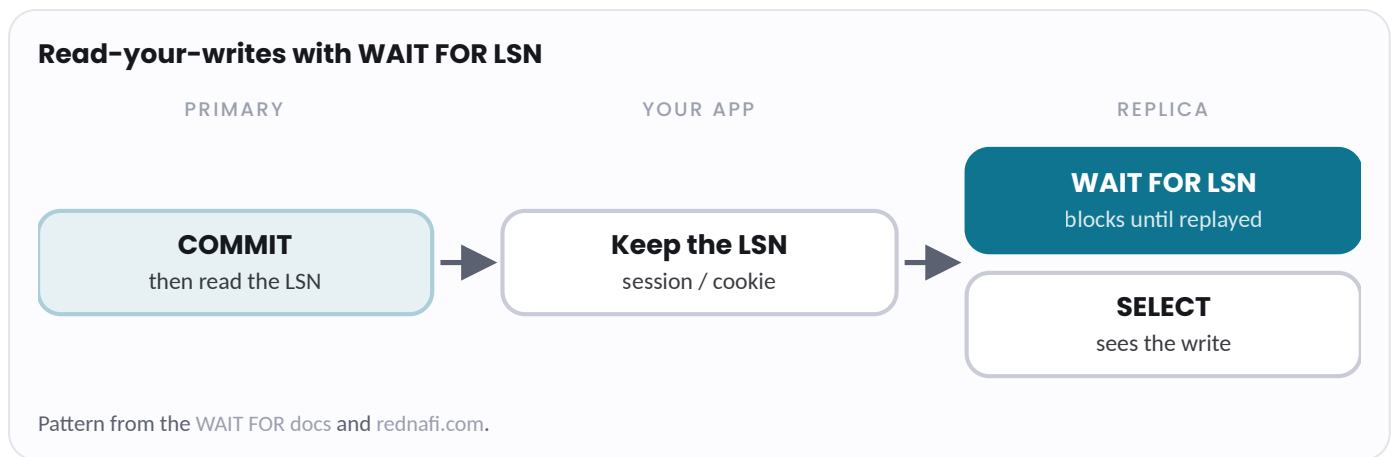
## 2. WAIT FOR LSN: read your own writes on a replica

The classic replica bug goes like this. A user saves their profile (a write to the primary), the page reloads, and the read goes to a [read replica](#) *≈ a standby server that replays the primary's write-ahead log and serves read-only queries that hasn't replayed the change yet*. The user sees their old data and clicks Save again.

Every change in Postgres has an address in the [write-ahead log \(WAL\)](#) *≈ the append-only log where Postgres records every change before touching data files; replicas stay in sync by replaying it*, called the [LSN](#) *≈ log sequence number: a byte position in the WAL that only ever increases, so "LSN ≥ X" means "has seen everything up to X"*. PostgreSQL 19 adds a command that blocks until a standby has reached a given LSN.<sup>11</sup>

```
-- 1. On the primary, after COMMIT, on the same connection:
SELECT pg_current_wal_insert_lsn(); -- e.g. 0/306EE20

-- 2. On the replica, before reading:
WAIT FOR LSN '0/306EE20' WITH (TIMEOUT '100ms', NO_THROW);
SELECT * FROM profiles WHERE user_id = 42;
```



The command returns `success`, `timeout` or `not in recovery` (if the server is not a standby, or was promoted while waiting). With `NO_THROW` it returns that status instead of raising an error.<sup>11</sup> The default `MODE` is `standby_replay`, which waits until the change is applied and visible. There are also `standby_write`, `standby_flush` and `primary_flush` modes.<sup>11</sup>

Two practical rules. First, read the LSN **after** you commit. If you capture it while the transaction is still open, your commit lands at a higher LSN and the wait can return too early.<sup>12</sup> Second, `WAIT FOR` must be a top-level command: not inside a function or a `DO` block, not while holding a snapshot, and not at isolation levels above `READ COMMITTED`.<sup>11</sup>

### 3. `pg_plan_advice`: query hints, finally (sort of)

Core Postgres has never had query hints *≈ instructions embedded with a query that tell the planner which join order or index to use, instead of letting it decide from statistics*. PostgreSQL 19 ships a contrib extension, `pg_plan_advice`, that lets you "stabilize and control" the planner's decisions, plus a companion `pg_stash_advice` that applies saved advice automatically based on the query.<sup>3</sup>

The neat part is that it can tell you what the planner already chose, in a small language you can feed back to it:<sup>13</sup>

```
LOAD 'pg_plan_advice';
EXPLAIN (COSTS OFF, PLAN_ADVICE)
  SELECT * FROM join_fact f JOIN join_dim d ON f.dim_id = d.id;
-- ...plan...
-- Generated Plan Advice:
--  JOIN_ORDER(f d)  HASH_JOIN(d)  SEQ_SCAN(f d)  NO_GATHER(f d)

SET pg_plan_advice.advice = 'JOIN_ORDER(f d)';
```

`EXPLAIN` then reports each piece of advice as `matched`, `partially matched`, `not matched`, `inapplicable`, `conflicting` or `failed`.<sup>13</sup> The documentation is frank about the risks. Advice stops the planner from adapting when your data changes. It can only choose among plans the planner already considers viable. And

bad advice can produce plans marked `Disabled: true` that are worse than no advice at all.<sup>13</sup> Treat it as a way to pin a known-good plan for a query that keeps regressing, not as a tuning tool for everyday use.

## 4. Smarter autovacuum

**Autovacuum** *≈ the background process that runs VACUUM and ANALYZE on tables automatically when enough rows have changed* gets two upgrades. It can now use parallel workers to vacuum a table's indexes ( `autovacuum_max_parallel_workers` , plus a per-table `autovacuum_parallel_workers` ). And a new **scoring system** decides which tables to process first, with weights you can tune ( `autovacuum_vacuum_score_weight` , `autovacuum_freeze_score_weight` and others). A new view, `pg_stat_autovacuum_scores` , shows each table's score.<sup>3</sup>

Why does order matter? The release notes say the goal is to prioritise "the tables that most need vacuuming or analyzing".<sup>3</sup> On a database with many tables, that means a table that urgently needs **freezing** *≈ VACUUM rewriting old transaction IDs in a table as "frozen", which has to happen before the transaction counter wraps around* no longer has to wait its turn behind tables that only need routine cleanup. The separate freeze and multixact-freeze weights let you tune exactly that.

## 5. Logical replication without the restart

**Logical replication** *≈ copying row-level changes (rather than raw WAL bytes) to another server, often for zero-downtime upgrades or feeding other systems* gets two long-requested features:<sup>3</sup>

- **Sequences are replicated** ( `CREATE PUBLICATION ... ALL SEQUENCES` , `ALTER SUBSCRIPTION ... REFRESH SEQUENCES` ). Until now, sequence values weren't carried over, so after a logical-replication cutover you had to sync them yourself before the new server could safely take inserts.
- **No restart to turn it on.** If `wal_level` is `replica` , logical replication can now be enabled without a server restart. A new read-only setting, `effective_wal_level` , reports the level actually in effect.

## 6. Performance work you get for free

The release notes list automatic scaling of the I/O worker pool for **asynchronous I/O** *≈ letting Postgres issue many disk reads at once and keep working while they complete, instead of waiting for each one* ( `io_min_workers` , `io_max_workers` , `io_worker_idle_timeout` , `io_worker_launch_interval` ), faster foreign-key checks, and regular table scans that can now mark pages all-visible in the **visibility map** *≈ a per-table bitmap recording which pages contain only rows visible to everyone, which makes index-only scans and VACUUM cheaper* (before, only `VACUUM` and `COPY ... FREEZE` could).<sup>3</sup> For the foreign-key work, the Beta 1 announcement claimed "2x better performance on inserts when foreign key checks are present". That's the project's own number, so measure it on your own workload.<sup>14</sup>

## WHAT IT MEANS FOR YOU

**New defaults you might not notice.** These quietly change behaviour on upgrade. All of them are from the official "Migration to Version 19" section.<sup>3</sup>

## Defaults that change when you move to PostgreSQL 19

### PostgreSQL 18

```
jit = on
default_toast_compression = pglz
log_lock_waits = off
max_locks_per_transaction = 64
standard_conforming_strings
    can be set to off
RADIUS authentication available
```

### PostgreSQL 19

```
jit = off
default_toast_compression = lz4
log_lock_waits = on
max_locks_per_transaction = 128
standard_conforming_strings
    is always on
RADIUS removed
```

From the PostgreSQL 19 release notes. Lock-table sizing also changed, so 128 in 19 holds about what 64 held before.

A few of these need a decision:

- **JIT** *≈ just-in-time compilation: turning parts of a query's expressions into machine code at run time, which can help long analytical queries but adds start-up cost* **is now off by default**. The release notes single out sites running many large analytical queries: they must now turn it back on themselves with the `jit` parameter. If that's you, test with `jit = on` and decide.<sup>3</sup>
- **TOAST** *≈ how Postgres stores large values (long text, big JSON) out of line, compressed and split into chunks* **now uses LZ4**. The project says LZ4 provides "better default compression and decompression performance".<sup>14,3</sup>
- **max\_locks\_per\_transaction** **doubled from 64 to 128**, but the notes say lock-table sizing changed, so values "must now be doubled to match their previous capacity". If you had set it to 256, set it to 512 in 19.<sup>3</sup>
- **standard\_conforming\_strings** **can no longer be turned off**. Dumps made by an older `pg_dump` with it off won't load correctly into 19. Take dumps with the 19 `pg_dump`.<sup>3</sup>
- **pg\_upgrade blockers**: clusters with `btree_gist` indexes on `inet` / `cidr` columns (those operator classes could silently skip matching rows), and databases, roles or tablespaces whose names contain carriage returns or line feeds, will refuse to upgrade until you fix them.<sup>3</sup>
- **MD5 passwords now trigger a warning** after a successful login (they were deprecated in 18), and there's a new password-expiry warning, 7 days ahead by default.<sup>3</sup>

**What to do this month.** If you run Postgres 14, its final release is on **12 November 2026**, so you need an upgrade plan anyway.<sup>5</sup> Whether to jump straight to 19 or go to 18 first is mostly a question of how soon your cloud provider offers 19 and how much testing time you have. Either way, run your test suite against Beta 4 or the RC now, check for the `pg_upgrade` blockers above, and review JIT if you run analytics.

## ☀ Watch out for

- **Blog posts written before September are out of date.** Many "What's new in Postgres 19" articles still show SQL/PGQ, `FOR PORTION OF` and online checksums. None of them is in 19.<sup>1</sup>
- **REPACK CONCURRENTLY is not a drop-in for every table.** It needs a primary key, a free replication slot and enough disk space, and it is not MVCC-safe.<sup>10</sup>
- **WAIT FOR LSN only compares numbers.** It "has no notion of which timeline" a WAL record belongs to, so after a failover an LSN taken on the old primary may not mean what you think.<sup>11</sup>
- **Betas aren't upgradable in place.** Going from a beta to the final release needs `pg_upgrade` or a dump and restore, like any major upgrade.<sup>1</sup>

## WHAT HAPPENS NEXT

The RC is due in early October and GA "may" follow in October.<sup>1</sup> The next scheduled minor release, on **12 November**, will be PostgreSQL 14's last.<sup>2,5</sup> The reverted features are now candidates for PostgreSQL 20, but nothing about them is promised yet. The bigger lesson is cultural: a database whose community will delete 47 commits of a headline feature three weeks before release is one that values your data over its own press release.

## 📖 Words from this story

**Major version:** a feature release (18, 19) that can change on-disk formats; upgrading needs `pg_upgrade` or dump/restore.

**Minor release:** a bug/security-fix release (18.6) installed by swapping binaries.

**Property graph queries (SQL/PGQ):** querying tables as nodes and edges with path patterns.

**Data checksum:** a per-page fingerprint that turns silent corruption into an error.

**Partitioned table:** one logical table stored as several child tables by key range or list.

**MVCC:** updates create new row versions; old ones stay for transactions that still need them.

**Dead tuples:** row versions no transaction can see any more.

**Bloat:** space a table or index uses beyond what its live rows need.

**ACCESS EXCLUSIVE lock:** the strongest table lock; blocks all reads and writes.

**Logical decoding:** turning WAL back into row-level changes.

**Replication slot:** a WAL bookmark that stops Postgres from deleting log a consumer still needs.

**Read replica:** a standby that replays WAL and serves read-only queries.

**Write-ahead log (WAL):** the append-only change log written before data files.

**LSN:** a byte position in the WAL; only ever increases.

**Query hints:** instructions that override the planner's choices.

**Autovacuum:** background VACUUM/ANALYZE triggered by table activity.

**Freezing:** VACUUM marking old transaction IDs as frozen before the counter wraps around.

**Logical replication:** copying row-level changes to another server.

**Asynchronous I/O:** issuing many reads at once instead of waiting for each.

**Visibility map:** a bitmap of pages whose rows are visible to everyone.

**JIT:** compiling query expressions to machine code at run time.

**TOAST:** out-of-line, compressed storage for large values.

# STORY 2 MongoDB 9.0: faster, stricter, and null means something new

MONGODB 9.0 · GA, ANNOUNCED 29 SEP 2026 · ~14 MIN READ

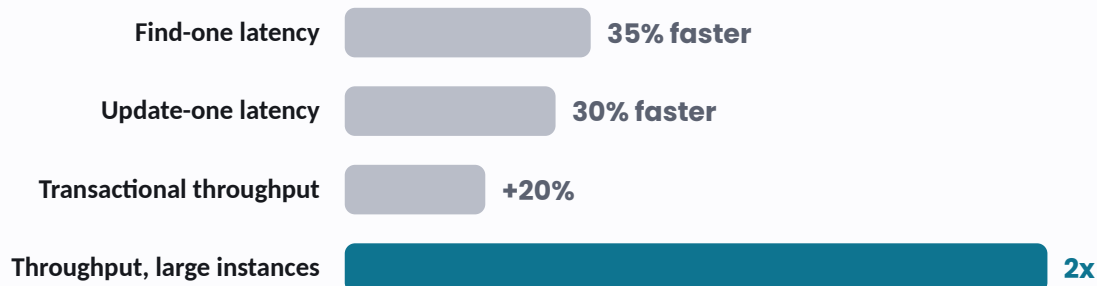
Picture a query that has run in your app for years: find orders where `items.discount` is `null`. You upgrade from MongoDB 8.0 to 9.0, change nothing, and the same query now returns more documents than it did yesterday. You haven't hit a bug. MongoDB 9.0 deliberately redefined what `null` means when a dotted path runs into an array. It's one line in the compatibility notes, sitting under a launch that headlined "up to 2x throughput". This story covers both: what's actually new in 9.0, and the small print to read before you upgrade.

## WHAT HAPPENED

MongoDB announced 9.0 on **29 September 2026** at an event at Nasdaq MarketSite in New York, alongside a public preview of a new Atlas architecture called **Atlas Infinite**.<sup>15</sup> The release notes list **9.0.0, 9.0.1 and 9.0.2 all dated 28 September**. 9.0.1 includes a fix for **CVE-2026-89099**, and **9.0.3 is "upcoming"**, with a fix for a known sharding hang described below.<sup>16</sup>

MongoDB calls it "the best version of MongoDB ever built".<sup>17</sup> Its own benchmarks, compared with MongoDB 8.0, claim up to **35% faster** find-one queries, up to **30% faster** update-one queries, up to **2x throughput on large instances** and up to **20% more throughput** for transactional workloads.<sup>15</sup> These are vendor numbers from "internal testing with public benchmarks", not independent results.<sup>17</sup>

### MongoDB's own claims: 9.0 vs 8.0 ("up to")



Vendor claims from MongoDB's press release and 9.0 page, based on internal testing. Bar length shows the claimed % gain (2x = +100%). No independent benchmarks yet.

9.0 is available on Atlas (MongoDB's managed cloud service), in Enterprise Advanced and in the free Community Edition. Like every major version, it gets five years of support.<sup>17,18</sup>

## THE BACKGROUND YOU NEED

**Major, minor and now "Feature Update" releases.** MongoDB changed its release model with this version, so here is a quick recap.

MongoDB ships **major releases** *≈ an X.0 version with new features and possibly backward-incompatible changes, supported for five years*, with **minor releases** *≈ interim X.Y versions such as 8.1, 8.2 and 8.3 that deliver new features between majors* in between. Starting with 8.2, self-managed users could also run the minor releases.<sup>18</sup> **9.0 changes the model**

again. From now on, minor releases go **only to Atlas clusters on Auto Upgrade**. Self-managed users get one annual **Feature Update** release, numbered **x.5** (so 9.5 would be the next one).<sup>18</sup> If you run MongoDB yourself, you'll see new features less often.

**FCV, the safety catch on upgrades.** MongoDB separates "running the new binaries" from "using features that change data on disk". The second is controlled by the **feature compatibility version (FCV)** *≈ a cluster-wide setting that keeps new on-disk formats and features switched off, so you can still downgrade, until you raise it*. You can only upgrade to 9.0 from **8.0 or 8.3**, with FCV set to **"8.0"**. If you're on 7.0, you have to go to 8.0 first. The upgrade is a rolling one: upgrade the secondaries one at a time, step down the primary, upgrade it, and only then run `setFeatureCompatibilityVersion: "9.0"`.<sup>19</sup> MongoDB recommends a "burn-in period" before raising FCV, because once you raise it, downgrading means first removing any 9.0-only features you've used.<sup>19</sup>

```
// Only after every member runs 9.0 binaries and you've watched it for a while:
db.adminCommand({ setFeatureCompatibilityVersion: "9.0", confirm: true })
```

## HOW IT WORKS: THE SIX CHANGES THAT MATTER

### 1. `null` on dotted paths now follows a simpler rule

In MongoDB, a **dotted path** *≈ a field reference such as "a.b" that reaches into embedded documents, and automatically into every element when it meets an array* like `"a.b"` walks into arrays implicitly. The awkward question has always been what `{ "a.b": null }` should match when `a` is an empty array, or an array of plain numbers with no `b` anywhere.

9.0's answer is simple: "a dotted path that does not resolve to a non-null value evaluates as `null`".<sup>16</sup> Here's the official example, before and after:<sup>20</sup>

#### Same data, same query, different answer

##### MongoDB 8.3 and earlier

```
{ _id: 1, a: [ 1 ] }
{ _id: 2, a: [ ] }

find({ "a.b": null })
→ neither document

find({ "a.b": { $ne: null } })
→ both documents
```

##### MongoDB 9.0

```
{ _id: 1, a: [ 1 ] }
{ _id: 2, a: [ ] }

find({ "a.b": null })
→ both documents

find({ "a.b": { $ne: null } })
→ neither document
```

From the 9.0 compatibility notes. Nested arrays such as `a: [[[b: 3]]]` change the same way.

The change affects `$eq`, `$ne`, `$in`, `$nin`, `$gte`, `$lte` and equality matching in `$lookup`. MongoDB's summary: after the upgrade, `{ $ne: null }` queries "return fewer documents" and `{ $eq: null }` queries "return more documents" when the path crosses an empty array, an array of scalars or a nested array.<sup>20</sup>

The new rule is easier to reason about. But the most common place you'll find `$ne: null` is an "only documents that have this field" filter, perhaps in a `$lookup` join or a report. Grep your code for `null` comparisons on dotted paths before you raise FCV.

## 2. Server-side JavaScript is back, in a sandbox

`$where`, `$function` and `$accumulator` let you run JavaScript inside the database. They had been deprecated. In 9.0 they're **no longer deprecated**, because MongoDB rebuilt them on a new JavaScript engine based on [WebAssembly \(WASM\)](#) *≈ a portable, sandboxed bytecode format; code inside it can only touch the memory and functions the host explicitly gives it*. MongoDB says this gives "stronger sandboxing and isolation than prior versions".<sup>16</sup>

There's a behaviour change hiding in this one. The new engine evaluates local `Date` operations **in UTC**, whatever the server's time zone. On a server set to New York time, `date.getHours()` for 13:30 UTC used to return `8`. In 9.0 it returns `13`.<sup>20</sup> The fix MongoDB recommends is to stop depending on the host's time zone and use aggregation operators with an explicit zone, such as `{ $hour: { date: "$occurredAt", timezone: "America/New_York" } }`.<sup>20</sup> And on `ppc64le` (IBM Power) servers, there's no JavaScript engine at all in 9.0. Those operations, and `mapReduce`, simply fail there.<sup>20</sup>

## 3. Guardrails: memory caps, transaction caps and overload control

The theme of 9.0 is **protecting the cluster from its own workload**. MongoDB pitches this for AI agents with "spiky, unpredictable traffic", but it helps any app that has one bad query.<sup>15</sup> There are three layers:

### Three layers of protection in MongoDB 9.0

#### Ingress queue (Atlas only)

throttle new operations under overload

#### Transaction cap

max 10,000 open multi-doc transactions

#### Per-operation memory cap

1 GB or 20% of RAM, whichever is larger

#### Per-stage memory cap

100 MB per stage, then spill to disk

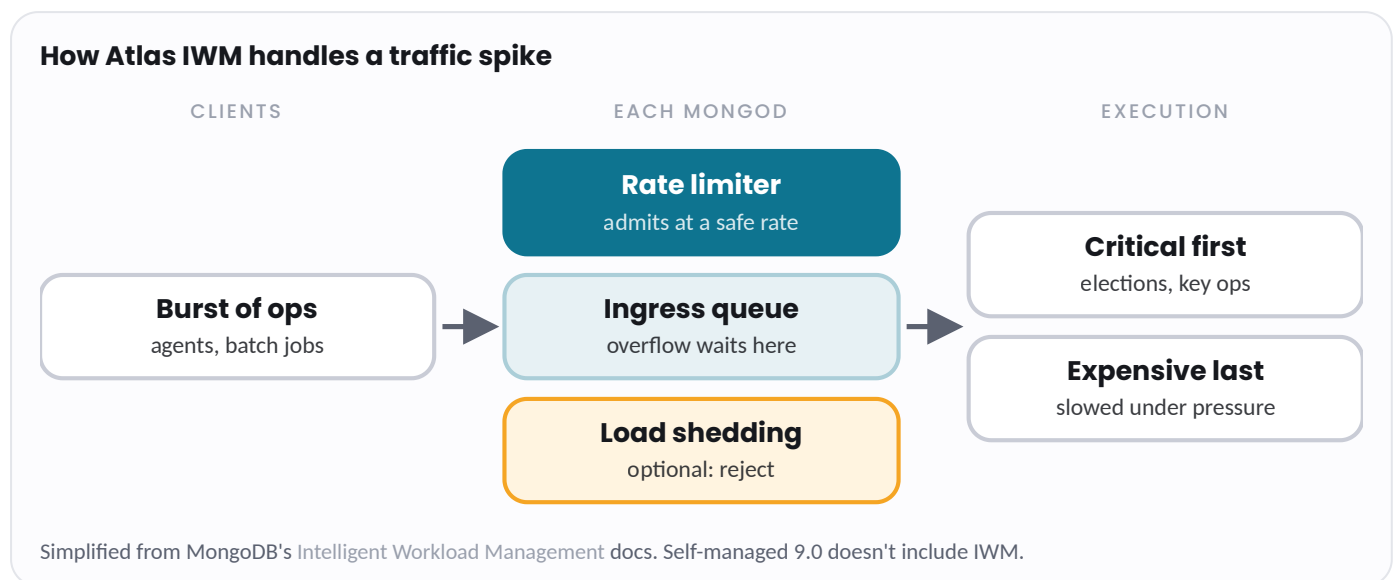
From the 9.0 release notes, aggregation limits and Intelligent Workload Management docs.

**Per-operation memory limit.** Aggregation stages have a **100 MB per-stage** memory limit. Past it, a stage either [spills to disk](#) *≈ writes intermediate data to temporary files instead of holding it in RAM; slower, but it finishes* (with `allowDiskUse`) or fails. 9.0 adds a cap on the **whole operation**: by default "1 gigabyte or 20% of the memory available to the server process, whichever is greater". An operation that exceeds it is ended with `QueryExceededMemoryLimitNoDiskUseAllowed` (if a stage could have spilled but `allowDiskUse` is false) or `ExceededMemoryLimit`.<sup>21</sup> A new `serverStatus` counter, `metrics.query.operationsFailedDueToMemoryLimit`,

tells you whether it's happening.<sup>16</sup> In plain terms: a pipeline with many medium-sized stages, each under 100 MB but together using several gigabytes, used to finish. Now it can be killed.

**Transaction cap.** The server now limits concurrently open [multi-document transactions](#) *≈ transactions that read or write several documents atomically, like a SQL transaction* from external clients. The `maxConcurrentMultiDocumentTransactions` parameter defaults to **10,000**, and new transactions beyond that get a `TooManyOpenTransactions` error.<sup>16</sup> Most apps never get close. An app that leaks open transactions will now find out quickly.

**Intelligent Workload Management (IWM)** is **Atlas-only** and active on dedicated clusters running 9.0.<sup>22</sup> Its core idea is [admission control](#) *≈ deciding how fast to let new work start, so running work isn't starved by a flood of new requests*. Each `mongod` runs an ingress rate limiter. When traffic exceeds capacity, excess operations wait in an **ingress queue** instead of competing for CPU and cache. Critical reads and writes and internal work (such as elections) run ahead of "background jobs and expensive queries".<sup>22</sup> Optionally, **Load Shedding** caps the queue and rejects the overflow with an overload error. It's off by default on 9.0, and queueing, prioritisation and Load Shedding need M30 or larger replica sets.<sup>22</sup>



Coinbase, quoted in the launch, says IWM gives it "confidence that our clusters will stay responsive during peak trading windows".<sup>15</sup> It's a quote, not a measurement, but it shows the use case: sudden spikes you can't scale for fast enough.

#### 4. Schema rules the database guarantees

MongoDB has had [schema validation](#) *≈ rules, usually written in JSON Schema, that inserts and updates must satisfy* for years, but with a loophole. The default `strict` level checks new writes, yet documents that were already invalid before you added the rule can stay in the collection.<sup>23</sup> 9.0 adds a new level, `constraint`, which "guarantees that every document in the collection satisfies the rules".<sup>16</sup>

To switch an existing collection to `constraint`, you first run `collMod` with `prepareConstraintValidationLevel: true`. That blocks `bypassDocumentValidation` and validator changes. Then you set `validationLevel: "constraint"`, and MongoDB scans **every** existing document. If one fails, the

whole change fails.<sup>23</sup> While at `constraint`, you can't edit the rules. To change them, you drop to `strict`, edit, and go back up (with another full scan). And the validation action must be `error` or `errorAndLog`, not `warn`.<sup>23</sup> For a team that has been writing "the field is always there" checks in application code, this is the database finally making that promise.

## 5. Per-query control over the planner

MongoDB already lets you store `query settings` *≈ stored per-query-shape instructions, such as which index to use, that the server applies to every query of that shape* with the `setQuerySettings` command. 9.0 extends them: you can pass a `querySettings` document **directly** on a single `find`, `distinct` or `aggregate` call, set a `maxTimeMS` timeout for a whole `query shape` *≈ a query with its literal values stripped out, so a filter on status "A" and the same filter on status "B" count as the same shape*, and override internal server parameters for one shape with `queryKnobs` (which needs FCV 9.0).<sup>16</sup>

You also get better visibility. `$queryStats` now covers `insert`, `update` and `delete` as well as reads, collected by default from a 1% sample of operations.<sup>16</sup> It's the same idea as Postgres's `pg_plan_advice`: when the planner gets one critical query wrong, you can pin that one query without touching the rest.

## 6. Under the hood: time series, encryption, BSON

- **Time series collections** are now stored as a single namespace of compressed `buckets` *≈ groups of measurements from the same source and time window, stored together in one document*, instead of being exposed as a view, and you can finally `renameCollection` them. Clusters with **more than 5,000 time series collections in one database** may see a slower upgrade and write-latency spikes.<sup>16</sup>
- **Queryable Encryption** *≈ client-side encryption where the server can run some queries on data it can't decrypt*: prefix, suffix and substring queries on encrypted strings are now GA. The GA format is **incompatible** with the 8.2 public preview, so preview users have to re-create the collection.<sup>20</sup>
- **BSON validation is on by default**. The server now validates every `BSON` *≈ Binary JSON, MongoDB's on-disk and wire format for documents* object it ingests, and `net.wireObjectCheck` is deprecated.<sup>16</sup> Separately, `mongocryptd` is deprecated in favour of the Automatic Encryption Shared Library.<sup>16</sup>

### WHAT IT MEANS FOR YOU

**If you're on Atlas**, your upgrade timing depends on your Atlas settings. Raising FCV is where behaviour changes, so do it deliberately. Check the IWM metrics ("Operation Rate Limiting: Queued operations") after the upgrade, because with reactive auto-scaling enabled, Atlas uses the queued-operation rate to decide when to scale up.<sup>22</sup>

**If you self-manage**, note the new release model. 9.0 is what you'll run until a 9.5 Feature Update, with no interim minors.<sup>18</sup> If you're on 7.0, you have to go through 8.0 first.<sup>19</sup>

**A pre-upgrade checklist, all from the official notes:**

1. Search for `null` / `$ne: null` / `$in: [null]` comparisons on dotted paths, including inside `$lookup`.<sup>20</sup>
2. Search server-side JavaScript for local `Date` methods (`getHours()`, `getDate()` and so on).<sup>20</sup>

3. Find heavy aggregations and test them against the new per-operation memory cap. Add `allowDiskUse: true` where spilling is acceptable.<sup>21</sup>
4. If your code matches on old 2dsphere error codes ( `16755` , `16756` and others), switch to `510 / 511`.<sup>20</sup>
5. Hand-written change-stream loops must handle the new resumable error `11602` ( `InterruptedDueToReplStateChange` ) after elections. Compliant drivers resume automatically.<sup>20</sup>
6. Wait for **9.0.3** if you run sharded clusters (see below).<sup>16</sup>

### 🌟 Watch out for

- **A known sharding hang.** SERVER-133518: a shard "can hang during step-up" if a chunk migration was interrupted during an earlier step-down. The shard then can't become primary, and primary reads and writes on it are unavailable. It's fixed in 9.0.3, which isn't out yet.<sup>16</sup>
- **Queries can now be killed mid-flight** with `QueryKilledError` if an indexed field becomes multikey `= holding an array, so the index stores one entry per array element` while the query runs. Re-run after the write completes.<sup>20</sup>
- **Cross-database `renameCollection` on replica sets** now holds an exclusive lock on both collections for the whole operation, blocking writes and DDL. That's safer (it prevents lost writes), but plan for the pause.<sup>20</sup>
- **IWM and the "spiky traffic" story are Atlas features.** A self-managed 9.0 gets the memory and transaction caps, but not the ingress queue.<sup>22</sup>
- **The headline numbers are MongoDB's own**, compared with 8.0, not with the 8.x minors.<sup>15</sup>

## WHAT HAPPENS NEXT

Watch for 9.0.3 and the SERVER-133518 fix.<sup>16</sup> On the cloud side, **Atlas Infinite**, which separates compute from storage, is in public preview **on AWS only**. MongoDB claims it cuts "time to scale by more than 96%" and delivers "189% more throughput per dollar than Atlas Core" (its own tests), with other clouds planned.<sup>15</sup> Independent 9.0 benchmarks and real migration reports will be the next thing worth reading. The first Feature Update under the new model should be 9.5, but MongoDB hasn't announced a date.

## Words from this story

**Major release:** an X.0 version with possibly incompatible changes and five years of support.

**Minor release:** an interim X.Y feature release; Atlas Auto Upgrade only from 9.0.

**Feature compatibility version (FCV):** the switch that enables new on-disk features; raise it last.

**Dotted path:** a field path like "a.b" that also walks into arrays.

**WebAssembly (WASM):** sandboxed portable bytecode that can only use what the host allows.

**Spill to disk:** writing intermediate data to temp files instead of RAM.

**Multi-document transaction:** an atomic read/write across several documents.

**Admission control:** limiting how fast new work starts so running work isn't starved.

**Schema validation:** rules that inserts and updates must satisfy.

**Query settings:** stored per-shape instructions such as index choice.

**Query shape:** a query with its literal values removed.

**Buckets:** groups of time-series measurements stored as one document.

**Queryable Encryption:** client-side encryption the server can still query.

**BSON:** MongoDB's binary document format.

**Multikey:** an indexed field that holds an array.

## On the radar

**Also happened, in one line each. Some will get a full deep dive in a future issue.**

- **Valkey 9.2.0-rc1:** the Redis fork's next release can take RDB snapshots without `fork()`, alongside ACL roles, a radix-tree "Path Hash" type and LZ4-compressed replication.<sup>24</sup>
- **DuckDB 2.0:** now scheduled for 21 October (tentative), with the "Quack" client-server protocol reaching 1.0. Alpha builds are out but "explicitly *not* production-ready".<sup>25,26</sup>
- **Pgpool-II security release (1 Oct):** 4.7.3 and four older branches fix 7 CVEs, five rated critical and mostly in the watchdog and failover code, and an upgrade is "strongly recommended".<sup>27</sup>
- **MySQL 26.10.0 Early Access (18 Sep):** adds a built-in MySQL REST Service component that serves HTTPS endpoints without a separate MySQL Router. Not for production.<sup>28</sup>
- **Databricks:** cross-engine attribute-based access control (ABAC) for Unity Catalog managed tables went GA on 1 Oct, and Lakeflow pipelines can now run as an account group (5 Oct).<sup>29</sup>
- **Apache Kafka 4.4.0:** the release plan (26 KIPs, including transactional state stores for Kafka Streams and dead-letter queues for share groups) targeted no earlier than 9 Sep, and we found no release announcement as of 5 Oct.<sup>30</sup>

1. [PostgreSQL 19 Beta 4 Released!](#), PostgreSQL Global Development Group, 24 Sep 2026.

2. [PostgreSQL Roadmap](#), postgresql.org.

3. [PostgreSQL 19 release notes \(draft\)](#), postgresql.org, "as of 2026-09-14".

4. [PostgreSQL 18 RC 1 Released!](#), postgresql.org, 4 Sep 2025.

5. [Versioning policy](#), postgresql.org.

6. [Two features just left PostgreSQL v19](#), Command Prompt, Sep 2026.

7. [6 Reverts for PostgreSQL 19](#), Command Prompt, Sep 2026.

8. [Postgres 18 New Default for Data Checksums and How to Deal with Upgrades](#), Crunchy Data.

9. [PostgreSQL 19: the REPACK command](#), dbi services, 14 Apr 2026.

10. [REPACK](#), PostgreSQL 19 documentation.
11. [WAIT FOR](#), PostgreSQL 19 documentation.
12. [Reading your own writes with WAIT FOR LSN in Postgres 19](#), rednafi.com.
13. [pg\\_plan\\_advice](#), PostgreSQL 19 documentation.
14. [PostgreSQL 19 Beta 1 Released!](#), postgresql.org, 4 Jun 2026.
15. [MongoDB Launches MongoDB 9.0, the Best Version Ever Built, and Atlas Infinite for AI-Scale Demand](#), MongoDB press release, 29 Sep 2026.
16. [Release Notes for MongoDB 9.0](#), MongoDB docs.
17. [MongoDB 9.0: The Strongest Foundation Yet](#), MongoDB.
18. [MongoDB Versions and Upgrade Paths](#), MongoDB docs.
19. [Upgrade a Replica Set from 8.0 to 9.0](#), MongoDB docs.
20. [Compatibility Changes in MongoDB 9.0](#), MongoDB docs.
21. [Aggregation Pipeline Limits](#), MongoDB docs.
22. [Intelligent Workload Management](#), MongoDB Atlas docs.
23. [Specify Validation Level for Existing Documents](#), MongoDB docs.
24. [Release 9.2.0-rc1](#), valkey-io/valkey on GitHub, 16 Sep 2026.
25. [Release Calendar](#), DuckDB.
26. [Try DuckDB v2.0-dev](#), DuckDB, 2 Sep 2026.
27. [Pgpool-II 4.7.3, 4.6.8, 4.5.13, 4.4.18 and 4.3.21 released](#), postgresql.org, 1 Oct 2026.
28. [Changes in MySQL 26.10.0 \(Early Access Release\)](#), MySQL release notes, 18 Sep 2026.
29. [Databricks release notes: October 2026](#), Databricks.
30. [Release Plan 4.4.0](#), Apache Kafka wiki.